

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 8”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-8 mata kuliah Struktur Data yang membahas algoritma sorting lanjutan, yaitu Shell Sort, Quick Sort, dan Merge Sort, serta visualisasi algoritma sorting menggunakan Graphical User Interface (GUI) dengan Java Swing. Praktikum ini melanjutkan materi sorting dasar pada pekan sebelumnya dengan mempelajari algoritma yang lebih efisien.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan laporan di masa yang akan datang.

Padang, 5 Juni, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Shell Sort	2
2.1.1 Teori.....	2
2.1.2 Langkah-langkah	2
2.1.3 Contoh Kode	2
2.1.4 Hasil Output	2
2.1.5 Analisis	2
2.2 Quick Sort	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	3
2.2.3 Contoh Kode	3
2.2.4 Hasil Output	3
2.2.5 Analisis	4
2.3 Merge Sort	4

2.2.1 Teori	4
2.2.2 Langkah-langkah	4
2.2.3 Contoh Kode	5
2.2.4 Hasil Output	5
2.2.5 Analisis	5
2.4 Bubble Sort GUI.....	6
2.2.1 Teori	6
2.2.2 Langkah-langkah	6
2.2.3 Contoh Kode	6
2.2.4 Hasil	7
2.2.5 Analisis	7
2.5 Merge Sort GUI.....	7
2.2.1 Teori	7
2.2.2 Langkah-langkah	7
2.2.3 Contoh Kode	7
2.2.4 Hasil	8
2.2.5 Analisis	8
2.6 Tugas Pekan 8	9
2.5.1. Analisis	9
BAB III KESIMPULAN	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma sorting lanjutan seperti Shell Sort, Quick Sort, dan Merge Sort memiliki efisiensi yang lebih baik dibandingkan algoritma sorting sederhana ($O(n^2)$). Praktikum pekan ini memfokuskan pada implementasi ketiga algoritma tersebut serta pembuatan visualisasi interaktif menggunakan GUI untuk memahami proses pengurutan secara langkah demi langkah.

1.2 Tujuan

1. Memahami konsep dan cara kerja Shell Sort, Quick Sort, dan Merge Sort.
2. Mengimplementasikan algoritma sorting lanjutan dalam bahasa Java.
3. Membandingkan performa ketiga algoritma sorting tersebut.
4. Mengembangkan aplikasi GUI untuk visualisasi Bubble Sort dan Merge Sort.
5. Memahami teknik divide and conquer serta optimasi sorting.

1.3 Manfaat

1. Mahasiswa memahami algoritma sorting yang lebih efisien untuk data berukuran besar.
2. Meningkatkan kemampuan dalam mengimplementasikan algoritma rekursif dan visualisasi GUI.
3. Memberikan pengalaman praktis dalam menggabungkan struktur data dengan antarmuka pengguna.

BAB II

PEMBAHASAN

2.1 Shell Sort

2.1.1 Teori

Shell Sort adalah penyempurnaan dari Insertion Sort yang menggunakan gap (jarak) untuk membandingkan dan mengurutkan elemen yang berjauhan terlebih dahulu sebelum melakukan insertion sort dengan gap = 1.

2.1.2 Langkah-langkah

1. Tentukan gap awal ($n/2$).
2. Lakukan insertion sort pada setiap sublist dengan jarak gap.
3. Kurangi gap hingga gap = 1.

2.1.3 Contoh Kode

```
public static void shellSort(int[] array) {
    int n = array.length;
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = array[i];
            int j = i;
            while (j > gap && array[j - gap] > temp) {
                array[j] = array[j - gap];
                j -= gap;
            }
            array[j] = temp;
        }
    }
}

public static void main(String[] args) {
    int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
    System.out.print("Sebelum: ");
    printArray(data);
    shellSort(data);
    System.out.print("Setelah (Shell Sort): ");
    printArray(data);
}

public static void printArray(int[] array) {
    for (int i = 0; i < array.length; i++) {
        System.out.print(array[i] + " ");
    }
}
```

2.1.4 Hasil Output

```
Console X
terminated> shellSort 2511533026 [Java Application] D:\eclipse\p
Sebelum: 3 10 4 6 8 9 7 2 1 5
Setelah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.1.5 Analisis

Shell Sort memiliki kompleksitas waktu yang lebih baik dari $O(n^2)$ tergantung urutan gap. Lebih efisien daripada Insertion Sort untuk data berukuran sedang.

2.2 Quick Sort

2.2.1 Teori

Quick Sort adalah algoritma divide and conquer yang memilih pivot, mempartisi array, dan secara rekursif mengurutkan subarray.

2.2.2 Langkah-langkah

1. Pilih pivot menggunakan Median-of-Three.
2. Partisi array sehingga elemen kiri pivot lebih kecil dan kanan lebih besar.
3. Rekursif pada subarray kiri dan kanan.

2.2.3 Contoh Kode

```
1 package kelas_2511533026;
2
3 public class QuickSort_2511533026 {
4     static void swap(int[] arr, int i, int j) {
5         int temp = arr[i];
6         arr[i] = arr[j];
7         arr[j] = temp;
8     }
9     // Fungsi partisi, pilih partisi nilai menggunakan Median of Three
10    static void partition(int[] arr, int low, int high) {
11        int mid = low + (high - low) / 3;
12        // Urutkan elemen low, mid, dan high
13        if (arr[low] > arr[mid]) {
14            swap(arr, low, mid);
15        }
16        if (arr[mid] > arr[high]) {
17            swap(arr, mid, high);
18        }
19        swap(arr, low, high);
20    }
21    static int partition(int[] arr, int low, int high) {
22        // Fungsi untuk memilih pivot
23        partition(arr, low, high);
24        int pivot = arr[high]; // Pilih elemen terakhir sebagai pivot
25        int i = low - 1;
26        for (int j = low; j < high; j++) {
27            // Jika elemen lebih kecil dari pivot maka simpan pivot
28            if (arr[j] < pivot) {
29                // Increment indeks elemen yang lebih kecil
30                swap(arr, i, j);
31            }
32        }
33        swap(arr, i + 1, high);
34        return (i + 1);
35    }
36 }
```

```
1 package kelas_2511533026;
2
3 public class ShellSort_2511533026 {
4     public static void shellSort(int[] a) {
5         int n = a.length;
6         int gap = n / 2;
7         while (gap > 0) {
8             for (int i = gap; i < n; i++) {
9                 int temp = a[i];
10                int j = i;
11                while (j > gap && a[j - gap] > temp) {
12                    a[j] = a[j - gap];
13                    j = j - gap;
14                }
15                a[j] = temp;
16            }
17            gap = gap / 2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int[] data = {10, 7, 8, 9, 1, 5};
23        System.out.println("Sebelum: ");
24        printArray(data);
25        shellSort_2511533026(data);
26        System.out.println("Setelah (Shell Sort): ");
27        printArray(data);
28    }
29
30    public static void printArray(int[] arr) {
31        for (int i : arr) System.out.print(i + " ");
32        System.out.println();
33    }
34 }
```

2.2.4 Hasil Output

```
Console X
<terminated> QuickSort_2511533026 [Java Application] D:\eclipse\
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort:
1 5 1 5 5 5
```

2.2.5 Analisis

Quick Sort memiliki rata-rata kompleksitas $O(n \log n)$, namun worst case $O(n^2)$ jika pivot tidak dipilih dengan baik. Penggunaan Median-of-Three membantu mengurangi risiko worst case.

2.3 Merge Sort

2.3.1 Teori

Merge Sort adalah algoritma divide and conquer yang membagi array menjadi dua, mengurutkan secara rekursif, kemudian menggabungkan (merge) kedua bagian yang sudah terurut.

2.3.2 Langkah-langkah

1. Bagi array menjadi dua bagian.
2. Urutkan kedua bagian secara rekursif.
3. Gabungkan kedua subarray yang sudah terurut.

2.3.3 Contoh Kode

```
1 package Pkham0_2511533026;
2
3 public class MergeSort_2511533026 {
4
5     void merge(int arr[], int l_3026, int m_3026, int r_3026) {
6
7         int m1 = m_3026 - l_3026 + 1;
8         int m2 = r_3026 - m_3026 + 1;
9
10        int L[] = new int[m1];
11        int R[] = new int[m2];
12
13        for (int i = 0; i < m1; ++i)
14            L[i] = arr[l_3026 + i];
15
16        for (int j = 0; j < m2; ++j)
17            R[j] = arr[m_3026 + 1 + j];
18
19        int i = 0, j = 0;
20        int k = l_3026;
21
22        while (i < m1 && j < m2) {
23            if (L[i] <= R[j]) {
24                arr[k] = L[i];
25                i++;
26            } else {
27                arr[k] = R[j];
28                j++;
29            }
30            k++;
31        }
32
33        while (i < m1) {
34            arr[k] = L[i];
35            i++;
36            k++;
37        }
38
39        while (j < m2) {
40            arr[k] = R[j];
41            j++;
42            k++;
43        }
44    }
45 }
```

```
46 void sort(int arr[], int l_3026, int r_3026) {
47
48     if (l_3026 < r_3026) {
49
50         int m_3026 = (l_3026 + r_3026) / 2;
51
52         sort(arr, l_3026, m_3026);
53
54         sort(arr, m_3026 + 1, r_3026);
55
56         merge(arr, l_3026, m_3026, r_3026);
57     }
58 }
59
60 static void printArray(int arr[]) {
61     int n = arr.length;
62
63     for (int i = 0; i < n; ++i)
64         System.out.print(arr[i] + " ");
65
66     System.out.println();
67 }
68
69 public static void main(String args[]) {
70
71     int arr[] = {12, 11, 13, 5, 6, 7};
72
73     System.out.println("Sebelum Terurut:");
74     printArray(arr);
75
76     MergeSort_2511533026 ob = new MergeSort_2511533026();
77     ob.sort(arr, 0, arr.length - 1);
78
79     System.out.println("\nSetelah Terurut Menggunakan Merge Sort:");
80     printArray(arr);
81 }
82 }
```

2.3.4 Hasil Output

```
Console X
<terminated> MergeSort_2511533026 [Java Application] D:\eclipse\
Sebelum Terurut:
12 11 13 5 6 7

Setelah Terurut Menggunakan Merge Sort:
5 6 7 11 12 13
```

2.3.5 Analisis

Merge Sort selalu memiliki kompleksitas $O(n \log n)$ baik best, average, maupun worst case. Stabil dan cocok untuk data berukuran besar, namun memerlukan ruang tambahan $O(n)$.

2.4 Bubble Sort GUI

2.4.1 Teori

Visualisasi GUI Bubble Sort menampilkan proses pengurutan langkah demi langkah dengan highlight warna dan log langkah.

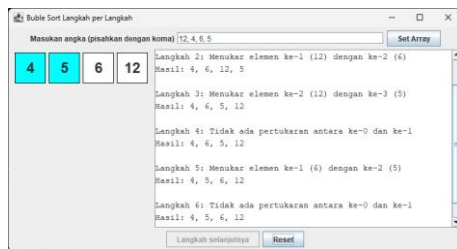
2.4.2 Langkah-langkah

1. Input array melalui JTextField.
2. Tombol "Langkah Selanjutnya" untuk simulasi satu pass Bubble Sort.
3. Update visual label dan log proses.

2.4.3 Contoh Kode

```
44 // JFileChooser untuk file
45 add(inputPanel_3026, BorderLayout.NORTH);
46 add(panelArray_3026, BorderLayout.CENTER);
47 add(controlPanel_3026, BorderLayout.SOUTH);
48 add(scrollPanel_3026, BorderLayout.EAST);
49
50 // event set array
51 setButton_3026.addActionListener(e -> setarrayFromInput_3026());
52 // event klik tombol
53 stepButton_3026.addActionListener(e -> performStep_3026());
54 // event reset
55 resetButton_3026.addActionListener(e -> reset_3026());
56
57 private void setarrayFromInput_3026() {
58     String text_3026 = inputField_3026.getText().trim();
59     if (text_3026.isEmpty()) return;
60     String[] parts_3026 = text_3026.split(",");
61     array_3026 = new int[parts_3026.length];
62     try {
63         for (int k = 0; k < parts_3026.length; k++) {
64             array_3026[k] = Integer.parseInt(parts_3026[k].trim());
65         }
66     } catch (NumberFormatException e) {
67         JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
68             + "yang dipisahkan koma", "Error", JOptionPane.ERROR_MESSAGE);
69     }
70     return;
71 }
72
73 l_3026 = 0;
74 l_3026 = 0;
75 stepCount_3026 = 1;
76 sorting_3026 = true;
77 stepButton_3026.setEnabled(true);
78 stepButton_3026.setEnabled(false);
79
80 panelArray_3026.removeAll();
81 labelArray_3026 = new JLabel(array_3026.length);
82
83 for (int k = 0; k < array_3026.length; k++) {
84     labelArray_3026[k] = new JLabel(String.valueOf(array_3026[k]));
85     labelArray_3026[k].setToolTipText("array[" + k + "] = " + array_3026[k]);
86     labelArray_3026[k].setOpaque(true);
87     labelArray_3026[k].setBackground(Color.WHITE);
88     labelArray_3026[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
89     labelArray_3026[k].setHorizontalAlignment(JLabel.CENTER);
90     labelArray_3026[k].setVerticalAlignment(JLabel.TOP);
91     panelArray_3026.add(labelArray_3026[k]);
92 }
93 panelArray_3026.repaint();
94
95 private void performStep_3026() {
96     if (l_3026 == array_3026.length - 1) {
97         sorting_3026 = false;
98         JOptionPane.showMessageDialog(this, "Sorting selesai!");
99         return;
100     }
101     resetHighlights_3026();
102     Stringbuilder stepLog_3026 = new Stringbuilder();
103     labelArray_3026[0].setBackground(Color.CYAN);
104     labelArray_3026[1].setBackground(Color.CYAN);
105     if (array_3026[l_3026] > array_3026[l_3026 + 1]) {
106         // swap
107         int temp_3026 = array_3026[l_3026];
108         array_3026[l_3026] = array_3026[l_3026 + 1];
109         array_3026[l_3026 + 1] = temp_3026;
110         labelArray_3026[l_3026].setBackground(Color.RED);
111         labelArray_3026[l_3026 + 1].setBackground(Color.RED);
112         stepLog_3026.append("Langkah ").append(stepCount_3026).append(" Menukar elemen ke-").
113             .append(l_3026).append(" ").append(array_3026[l_3026 + 1]).append(" dengan ke-").
114             .append(l_3026 + 1).append(" ").append(array_3026[l_3026]).append(" ke-").
115             .append(l_3026 + 1).append(" ").append(array_3026[l_3026 + 1]).append(" ke-").
116             .append(l_3026).append(" ").append(array_3026[l_3026]).append(" ke-").append("\n");
117     } else {
118         stepLog_3026.append("Langkah ").append(stepCount_3026).append(" Tidak ada pertukaran antara ke-").
119             .append(l_3026).append(" ").append(l_3026 + 1).append(" ke-").append("\n");
120     }
121     stepLog_3026.append("Selesai: ").append(arrayToString_3026(array_3026)).append("\n");
122     stepLog_3026.append(stepLog_3026.toString());
123     updateLabel_3026();
124     l_3026++;
125 }
126
127 private void reset_3026() {
128     inputField_3026.setText("");
129     panelArray_3026.removeAll();
130     panelArray_3026.repaint();
131     stepButton_3026.setEnabled(true);
132     stepButton_3026.setEnabled(false);
133     l_3026 = 0;
134     l_3026 = 0;
135     stepCount_3026 = 1;
136 }
137
138 private String arrayToString_3026(int[] arr) {
139     Stringbuilder sb_3026 = new Stringbuilder();
140     for (int k = 0; k < arr.length; k++) {
141         sb_3026.append(arr[k]);
142         if (k < arr.length - 1) sb_3026.append(", ");
143     }
144     return sb_3026.toString();
145 }
```

2.4.4 Hasil Output



2.4.5 Analisis

GUI ini sangat membantu dalam memahami proses Bubble Sort. Penggunaan warna (Cyan untuk perbandingan, Red untuk swap) membuat visualisasi menjadi lebih interaktif dan edukatif.

2.5 Merge Sort GUI

2.5.1 Teori

Visualisasi Merge Sort menggunakan queue untuk menyimpan tahapan merge dan menampilkan proses penggabungan secara bertahap.

2.5.2 Langkah-langkah

1. Input array dan generate semua tahap merge.
2. Proses merge secara bertahap dengan highlight.
3. Tampilkan log dan update array visual.

2.5.3 Contoh Kode

[illegible]

[illegible]

2.5.4 Hasil

Merge Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma) 5.3.12

3 5 12

Set Array

Langkah 1: Mula: merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mula: merge dari 0 ke 2
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Bandingkan dan salin elemen
 Langkah 9: Salin sisa kanan
 Langkah 10: Tempelkan ke array utama
 Langkah 11: Tempelkan ke array utama
 Langkah 12: Tempelkan ke array utama Selesai.

Langkah selanjutnya Reset

2.5.5 Analisis

Implementasi GUI Merge Sort cukup kompleks karena sifat rekursifnya, namun berhasil divisualisasikan dengan baik menggunakan queue dan state management.

2.6 Tugas Pekan 8

2.6.1 Analisis Kode Program

1. Deskripsi Program

Program ini merupakan implementasi algoritma Quick Sort untuk mengurutkan data playlist lagu. Program terdiri dari dua kelas utama:

- Lagu_2511533026: Kelas ADT yang merepresentasikan objek lagu dengan atribut judul_3026, penyanyi_3026, dan durasi_3026.
- Sorting_2511533026: Kelas utama yang mengelola array dataLagu_3026 (maksimal 20 lagu), melakukan input data, proses sorting, dan menampilkan hasil.

Program memenuhi semua aturan penamaan yang ditentukan, yaitu menggunakan akhiran NIM lengkap pada nama kelas dan akhiran _3026 pada seluruh variabel serta method.

2. Alasan Pemilihan Quick Sort

Saya memilih **Quick Sort** karena:

- Memiliki rata-rata kompleksitas waktu $O(n \log n)$ yang sangat efisien untuk data dalam jumlah sedang hingga besar.
- Algoritma *in-place*, sehingga tidak memerlukan array tambahan yang besar (hemat memori).
- Cocok untuk mengurutkan data playlist lagu berdasarkan durasi (numeric), sesuai salah satu persyaratan tugas.
- Quick Sort merupakan salah satu algoritma sorting paling populer dan banyak digunakan di dunia nyata (misalnya di sistem operasi dan database).

3. Analisis Cara Kerja Program

a. Struktur Data Data lagu disimpan dalam array bertipe Lagu_2511533026[]. Penggunaan array sesuai dengan permintaan tugas (bukan LinkedList).

b. Quick Sort Implementation

- Method utama: quickSort_3026()
- Menggunakan pendekatan Lomuto Partition Scheme.

- Pivot diambil dari elemen terakhir (high).
- Proses partition membagi array menjadi dua bagian: elemen yang lebih kecil dan lebih besar dari pivot.
- Rekursi dilakukan pada sub-array kiri dan kanan pivot.

c. Method Penting

- inputData_3026(): Mengisi data awal sebanyak 9 lagu (melebihi minimal 7 lagu yang diminta).
- tampilData_3026(): Menampilkan data sebelum dan sesudah sorting dengan format yang rapi.
- partition_3026(): Inti dari Quick Sort yang melakukan pembagian array.

```

// Metode inputData_3026
public static void inputData_3026() {
    printLagu_3026();
    printDurasi_3026();
    printPenyanyi_3026();
    printDurasi_3026(); // dalam detik
}

// Metode tampilData_3026
public static void tampilData_3026() {
    printLagu_3026();
    printDurasi_3026();
    printPenyanyi_3026();
    printDurasi_3026();
}

// Metode partition_3026
public static int partition_3026(int low, int high) {
    int pivot = dataLagu_3026[high].getDurasi();
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (dataLagu_3026[j].getDurasi() < pivot) {
            // swap
            // swap data
            dataLagu_3026[j] = dataLagu_3026[i];
            dataLagu_3026[i] = dataLagu_3026[j];
        }
    }
    // swap pivot
    dataLagu_3026[high] = dataLagu_3026[i];
    dataLagu_3026[i] = dataLagu_3026[high];
    return i;
}

// Metode main
public static void main(String[] args) {
    Sorting_2511533026 program = new Sorting_2511533026();
    program.inputData_3026();
    program.tampilData_3026();
    program.quickSort_3026();
    program.tampilData_3026();
    System.out.println("Sorting selesai. Terima kasih.");
}

```

```

// Metode tampilData_3026
public static void tampilData_3026() {
    printLagu_3026();
    printDurasi_3026();
    printPenyanyi_3026();
    printDurasi_3026();
}

// Metode partition_3026
public static int partition_3026(int low, int high) {
    int pivot = dataLagu_3026[high].getDurasi();
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (dataLagu_3026[j].getDurasi() < pivot) {
            // swap
            // swap data
            dataLagu_3026[j] = dataLagu_3026[i];
            dataLagu_3026[i] = dataLagu_3026[j];
        }
    }
    // swap pivot
    dataLagu_3026[high] = dataLagu_3026[i];
    dataLagu_3026[i] = dataLagu_3026[high];
    return i;
}

// Metode main
public static void main(String[] args) {
    Sorting_2511533026 program = new Sorting_2511533026();
    program.inputData_3026();
    program.tampilData_3026();
    program.quickSort_3026();
    program.tampilData_3026();
    System.out.println("Sorting selesai. Terima kasih.");
}

```

```

Console X
<terminated> Sorting_2511533026 [Java Application] D:\eclipse\plugins\...
=== Sorting Playlist NIM: 2511533026 ===
Data awal berhasil diinput (9 lagu)

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - Mina (270 detik)
2. La Rumba Del Perdon - Gipsy Kings (252 detik)
3. La Perla - Gipsy Kings (196 detik)
4. Besame Mucho - Andrea Bocelli (320 detik)
5. Shape of You - Ed Sheeran (233 detik)
6. Perfect - Ed Sheeran (263 detik)
7. Thinking Out Loud - Ed Sheeran (295 detik)
8. All of Me - John Legend (269 detik)
9. Photograph - Ed Sheeran (259 detik)

Data Setelah Quick Sort (Durasi Asc):
1. La Perla - Gipsy Kings (196 detik)
2. Shape of You - Ed Sheeran (233 detik)
3. La Rumba Del Perdon - Gipsy Kings (252 detik)
4. Photograph - Ed Sheeran (259 detik)
5. Perfect - Ed Sheeran (263 detik)
6. All of Me - John Legend (269 detik)
7. Mio Cristo Piange Diamanti - Mina (270 detik)
8. Thinking Out Loud - Ed Sheeran (295 detik)
9. Besame Mucho - Andrea Bocelli (320 detik)

Sorting selesai. Terima kasih.

```

BAB III

KESIMPULAN

Praktikum pekan ke-8 ini telah berhasil memberikan pemahaman yang komprehensif mengenai algoritma sorting lanjutan yaitu Shell Sort, Quick Sort, dan Merge Sort beserta visualisasinya dalam bentuk aplikasi GUI. Melalui implementasi dan visualisasi, kami dapat melihat perbedaan pendekatan serta keunggulan masing-masing algoritma dalam mengurutkan data. Penggunaan GUI pada Bubble Sort dan Merge Sort sangat membantu dalam memahami proses internal algoritma secara langkah demi langkah.

Secara keseluruhan, praktikum ini memperkuat kemampuan kami dalam mengimplementasikan algoritma yang lebih efisien serta mengintegrasikannya dengan antarmuka grafis. Pengetahuan yang diperoleh akan menjadi dasar yang kuat untuk mempelajari topik algoritma yang lebih kompleks pada pertemuan berikutnya.

DAFTAR PUSTAKA

1. Cormen, T. H., et al. (2009). *Introduction to Algorithms*. MIT Press.
2. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
4. Materi Kuliah Struktur Data – Universitas Andalas.
5. Dokumentasi Java Swing – Oracle.